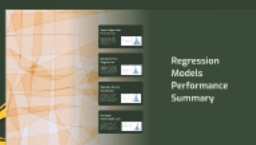


Modeling Fuel Efficiency: A Comparative Analysis of Linear and Ensemble Regression Techniques

An analysis of regression models to evaluate vehicle fuel efficiency, focusing on methodologies and outcomes.



Modeling Fuel Efficiency: A Comparative Analysis

I am Douglas Fulcher, presenting my project for the Applied Data Science Capstone course at Bellevue University. This project focuses on building and comparing various regression models to predict vehicle miles per gallon (MPG). My goal is to determine which modeling approach is most effective while demonstrating a scalable Python framework for testing and optimizing models.



Business Problem & Research Questions

Understanding Fuel Efficiency Factors

The primary business challenge is to understand the factors affecting vehicle fuel efficiency and leverage predictive modeling to enhance vehicle design, which is essential in the context of rising fuel costs and stringent environmental regulations.



Key Research Questions

This study aims to address five key research questions that will guide the exploration of vehicle traits and predictive modeling outcomes.

- Which type of regression model predicts MPG most accurately?
- How well do these models generalize to unseen data?
- Which vehicle traits — like weight or horsepower — have the greatest impact?
- How does interpretability change as models become more complex?
- And finally, what are the trade-offs between accuracy, transparency, and computation time?



Understanding Fuel Efficiency Factors

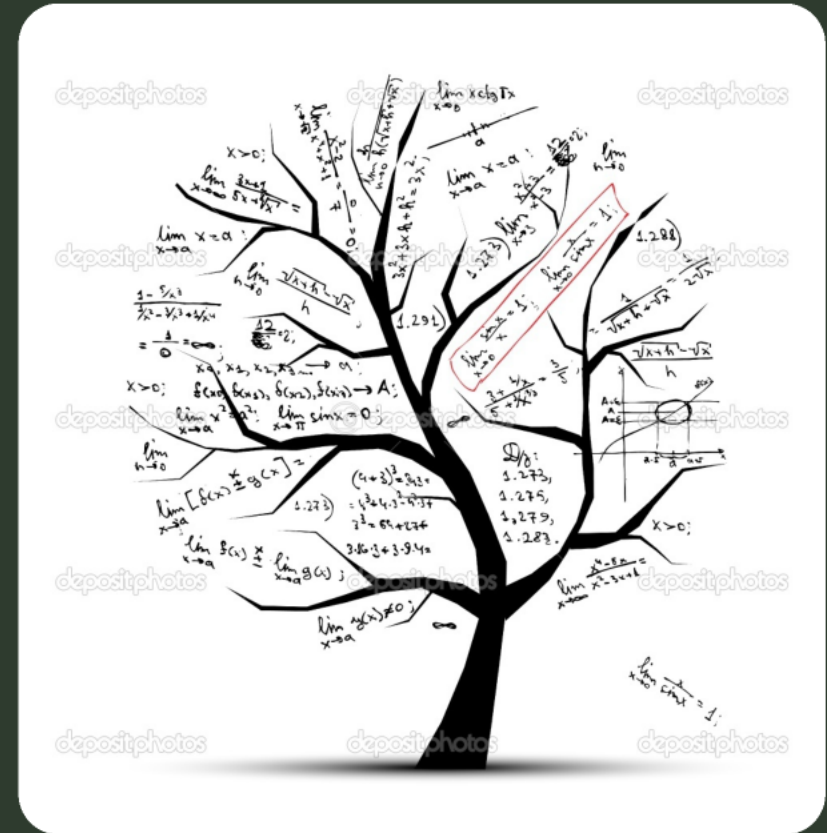
The primary business challenge is to understand the factors affecting vehicle fuel efficiency and leverage predictive modeling to enhance vehicle design, which is essential in the context of rising fuel costs and stringent environmental regulations.



Key Research Questions

This study aims to address five key research questions that will guide the exploration of vehicle traits and predictive modeling outcomes.

- Which type of regression model predicts MPG most accurately?
- How well do these models generalize to unseen data?
- Which vehicle traits — like weight or horsepower — have the greatest impact?
- How does interpretability change as models become more complex?
- And finally, what are the trade-offs between accuracy, transparency, and computation time?



Auto MPG Dataset Overview

The Auto MPG dataset consists of 398 observations of vehicles produced between 1970 and 1982. The dataset contains various features related to vehicle specifications, enabling analysis of their impact on fuel efficiency.

Key Features of the Dataset

Key features of the dataset include: weight, horsepower, engine displacement, number of cylinders, acceleration, model year, and origin. These attributes provide critical insights into how different vehicle characteristics affect miles per gallon (MPG).

Data Preprocessing Steps

Data preprocessing steps included handling missing values through mean imputation and applying one-hot encoding to categorical variables. This ensured that all data points were usable for model training and analysis.



Dataset & Preprocessing

Auto MPG Dataset Overview

The Auto MPG dataset consists of 398 observations of vehicles produced between 1970 and 1982. The dataset contains various features related to vehicle specifications, enabling analysis of their impact on fuel efficiency.

Key Features of the Dataset

Key features of the dataset include: weight, horsepower, engine displacement, number of cylinders, acceleration, model year, and origin. These attributes provide critical insights into how different vehicle characteristics affect miles per gallon (MPG).

Data Preprocessing Steps

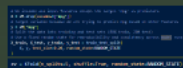
Data preprocessing steps included handling missing values through mean imputation and applying one-hot encoding to categorical variables. This ensured that all data points were usable for model training and analysis.

Modeling Framework & Code Design

Highlighting the automation and structure of the modeling code for efficiency and clarity.

Initial Setup

Established the foundational code structure, focusing on flexibility and ease of use through configuration dictionaries.



Configuration Dictionaries

Utilized Python dictionaries to define model types, hyperparameters, and evaluation metrics for streamlined model experimentation.



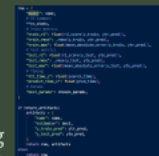
Automation

Implemented automation features that allow for rapid testing of multiple models with minimal code changes, enhancing experimentation efficiency.



Metrics Capture

Developed a structured logging system to capture performance metrics, enabling easy comparison of model results across configurations.



Initial Setup

```
# We include all input features except the target 'mpg' as predictors
X = df.drop(columns=["mpg"])
# Target variable because we are trying to predict mpg based on other features
y = df["mpg"]
# Split the data into training and test sets (80% train, 20% test)
# Use a fixed random state for reproducibility and consistency across model runs
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE
)

cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
```

Established the foundational code structure, focusing on flexibility and ease of use through configuration dictionaries.

Configuration Dictionaries

Utilized Python dictionaries to define model types, hyperparameters, and evaluation metrics for streamlined model experimentation.

```
pipelines = {  
    # Best suited for simple regression tasks where data distribution is expected to follow a linear trend.  
    # This model provides speed and interpretability ideal for understanding the impact of each feature.  
    # Linear Regression models may underperform on non-linear relationships unless polynomial or  
    # interaction terms are included.  
    "Linear Regression": Pipeline([  
        ("scaler", StandardScaler()),  
        ("model", LinearRegression())  
    ]),  
    # Decision Tree Regressor splits data into  
    # interpretable and easy to visualize. The  
    # without the need for scaling. This model  
    # depth and minimum sample splits are applied  
    # relationships between features need to be  
    "Decision Tree": Pipeline([  
        ("model", DecisionTreeRegressor(random_  
    ]),  
    # This model reduces the risk of overfitting  
    # trees' results. Random Forest can handle  
    # importance evaluation. The model is effective  
    # continuous variable.  
    "Random Forest": Pipeline([  
        ("model", RandomForestRegressor(random_  
    ]),  
    # Gradient Boosting Regressor builds sequentially  
    # made by the previous ones. This model is  
    # non-linear relationships. The model can  
    # as learning rate and number of trees.  
    "Gradient Boosting": Pipeline([  
        ("model", GradientBoostingRegressor(random_state=RANDOM_STATE))  
    ]),  
}  
  
param_grids = {  
    "Linear Regression": None, # no tuning needed for baseline  
    "Decision Tree": {  
        "model__max_depth": [None, 6, 10, 16],  
        "model__min_samples_leaf": [1, 2, 5],  
    },  
    "Random Forest": {  
        "model__n_estimators": [200],  
        "model__max_depth": [None, 10, 20],  
        "model__min_samples_leaf": [1, 2, 5],  
        "model__max_features": ["sqrt", "log2", 0.7],  
    },  
    "Gradient Boosting": {  
        "model__n_estimators": [200, 300],  
        "model__learning_rate": [0.05, 0.1],  
        "model__max_depth": [2, 3, 4],  
        "model__subsample": [0.7, 1.0],  
    },  
}
```

Automation

```
rows = []  
artifacts = []  
  
for name, pipe in pipelines.items():  
    r, a = evaluate_model(name, pipe, param_grids[name], X_train, y_train, X_test, y_test, return_artifacts=True)  
    rows.append(r)  
    artifacts.append(a)
```

Implemented automation features that allow for rapid testing of multiple models with minimal code changes, enhancing experimentation efficiency.

Metrics Capture

Developed a structured logging system to capture performance metrics, enabling easy comparison of model results across configurations.

```
row = {
    "model": name,
    # CV summary
    **cv_stats,
    # Train metrics
    "train_r2": float(r2_score(y_train, ytr_pred)),
    "train_rmse": _rmse(y_train, ytr_pred),
    "train_mae": float(mean_absolute_error(y_train, ytr_pred)),
    # Test metrics
    "test_r2": float(r2_score(y_test, yte_pred)),
    "test_rmse": _rmse(y_test, yte_pred),
    "test_mae": float(mean_absolute_error(y_test, yte_pred)),
    # Timing
    "fit_time_s": float(search_time),
    "predict_time_s": float(pred_time),
    # Params
    "best_params": chosen_params,
}

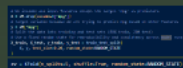
if return_artifacts:
    artifacts = {
        "name": name,
        "estimator": best,
        "y_train_pred": ytr_pred,
        "y_test_pred": yte_pred,
    }
    return row, artifacts
else:
    return row
```

Modeling Framework & Code Design

Highlighting the automation and structure of the modeling code for efficiency and clarity.

Initial Setup

Established the foundational code structure, focusing on flexibility and ease of use through configuration dictionaries.



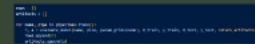
Configuration Dictionaries

Utilized Python dictionaries to define model types, hyperparameters, and evaluation metrics for streamlined model experimentation.



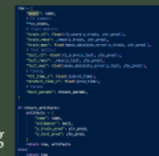
Automation

Implemented automation features that allow for rapid testing of multiple models with minimal code changes, enhancing experimentation efficiency.



Metrics Capture

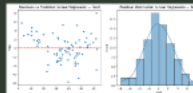
Developed a structured logging system to capture performance metrics, enabling easy comparison of model results across configurations.



Regression Models Performance Summary

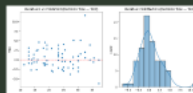
Linear Regression Performance

Linear Regression served as a baseline model with an R^2 of 0.845 and an RMSE of 2.89, demonstrating decent predictive capabilities but limited by its linear assumptions.



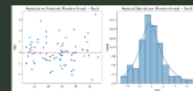
Decision Tree Regression

The Decision Tree Regressor achieved an R^2 of 0.876 and an RMSE of 2.58, showcasing improved accuracy thanks to its ability to model nonlinear relationships in the data.



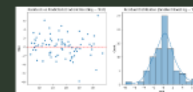
Random Forest Excellence

The Random Forest Regressor excelled with an R^2 score of 0.915 and an RMSE of 2.14, demonstrating its effectiveness in handling complex datasets while minimizing overfitting.



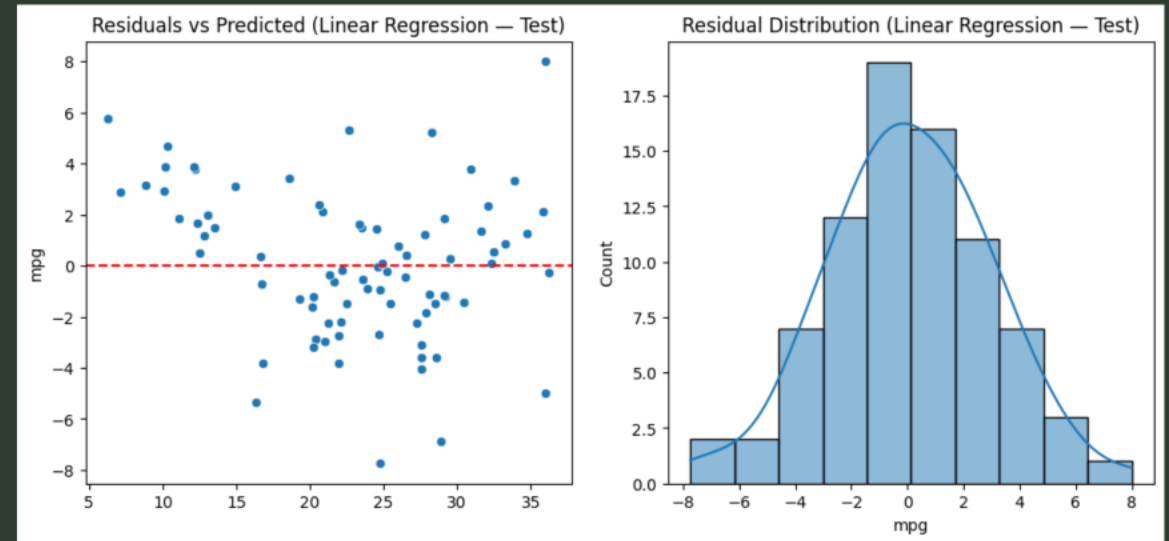
Gradient Boosting Results

Gradient Boosting yielded an R^2 of 0.895 and an RMSE of 2.37, providing a good balance of speed and accuracy, but it required careful hyperparameter tuning to optimize performance.



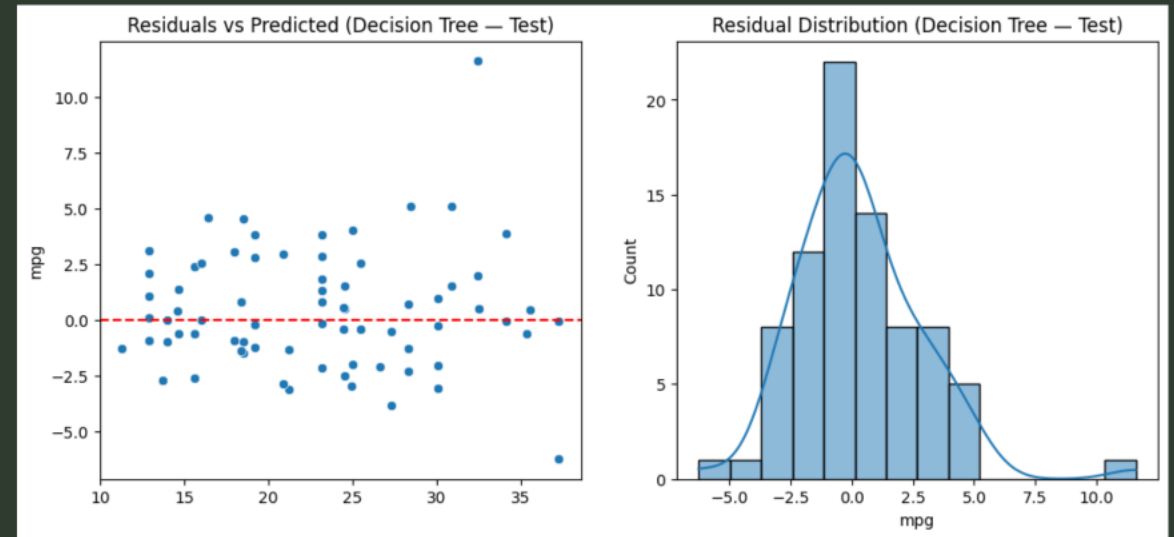
Linear Regression Performance

Linear Regression served as a baseline model with an R^2 of 0.845 and an RMSE of 2.89, demonstrating decent predictive capabilities but limited by its linear assumptions.



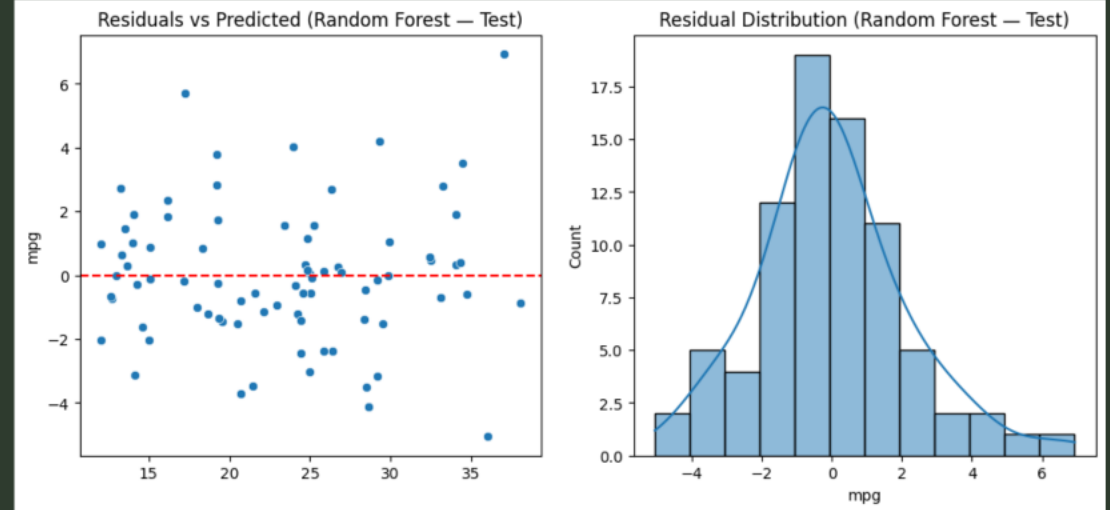
Decision Tree Regression

The Decision Tree Regressor achieved an R^2 of 0.876 and an RMSE of 2.58, showcasing improved accuracy thanks to its ability to model nonlinear relationships in the data.



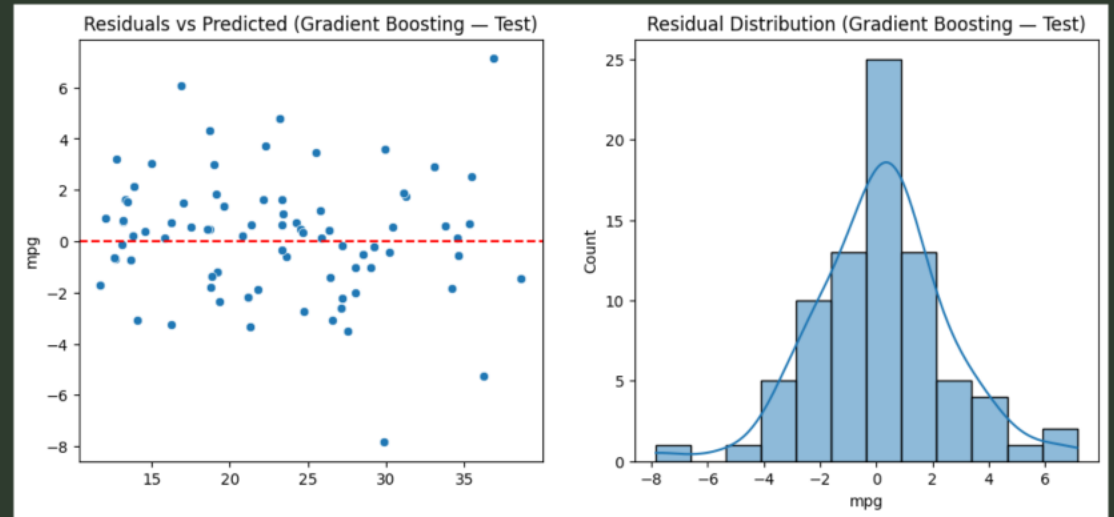
Random Forest Excellence

The Random Forest Regressor excelled with an R^2 score of 0.915 and an RMSE of 2.14, demonstrating its effectiveness in handling complex datasets while minimizing overfitting.



Gradient Boosting Results

Gradient Boosting yielded an R^2 of 0.895 and an RMSE of 2.37, providing a good balance of speed and accuracy, but it required careful hyperparameter tuning to optimize performance.



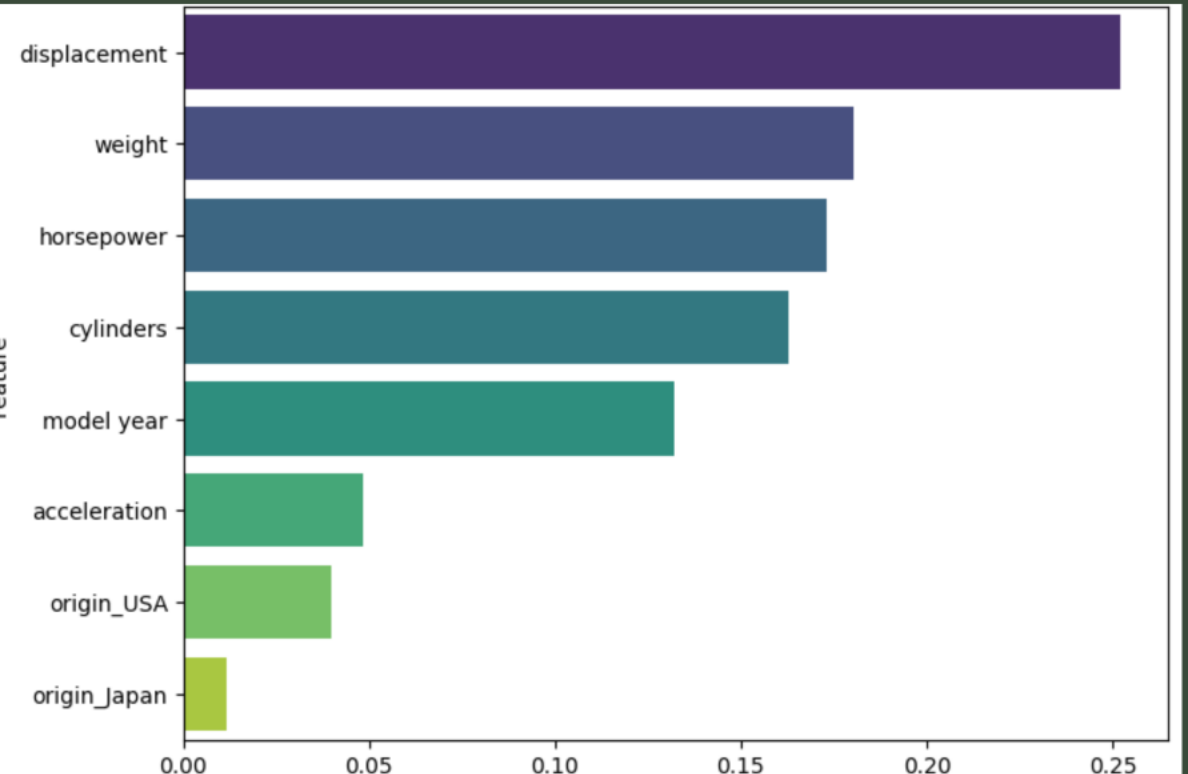
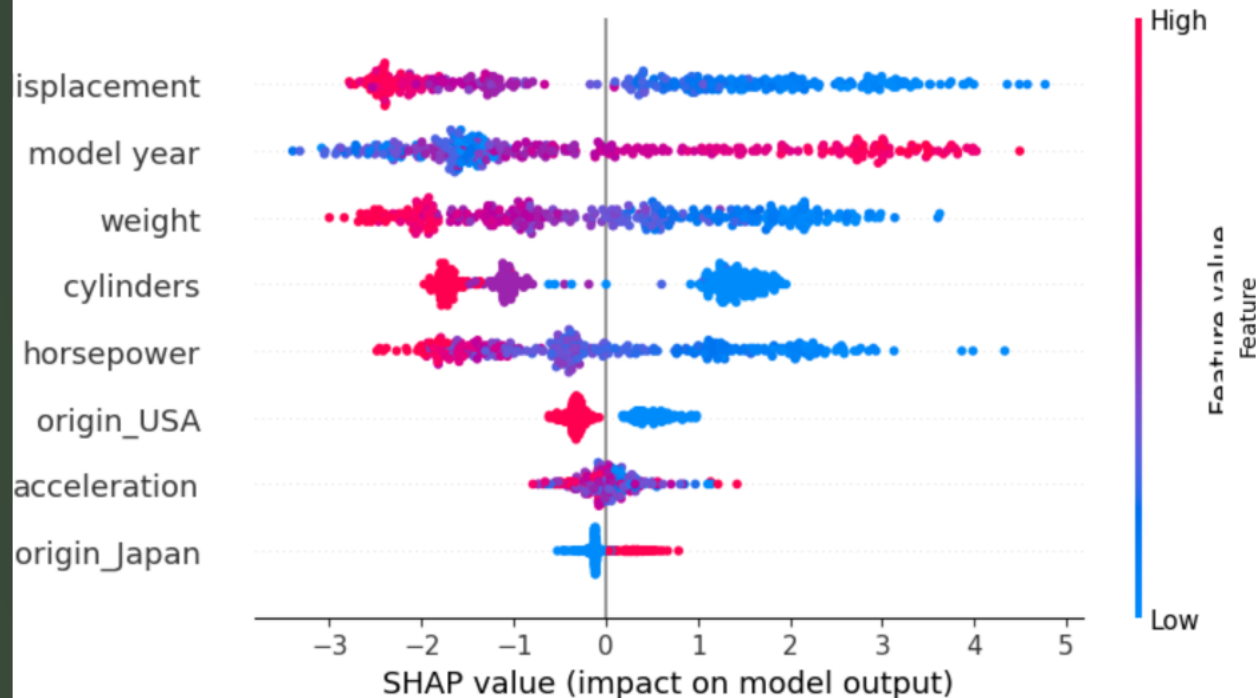
SHAP Analysis: Key Insights

SHAP analysis, or SHapley Additive Explanations, provides a detailed understanding of the influence of each feature on the model's predictions. Key insights reveal that displacement, weight, model year, and horsepower are influential features in the model. Higher displacement and weight values result in lower MPG. However, new model years with lower displacement and weight values result in higher MPG.

Traditional Metrics: Limitations

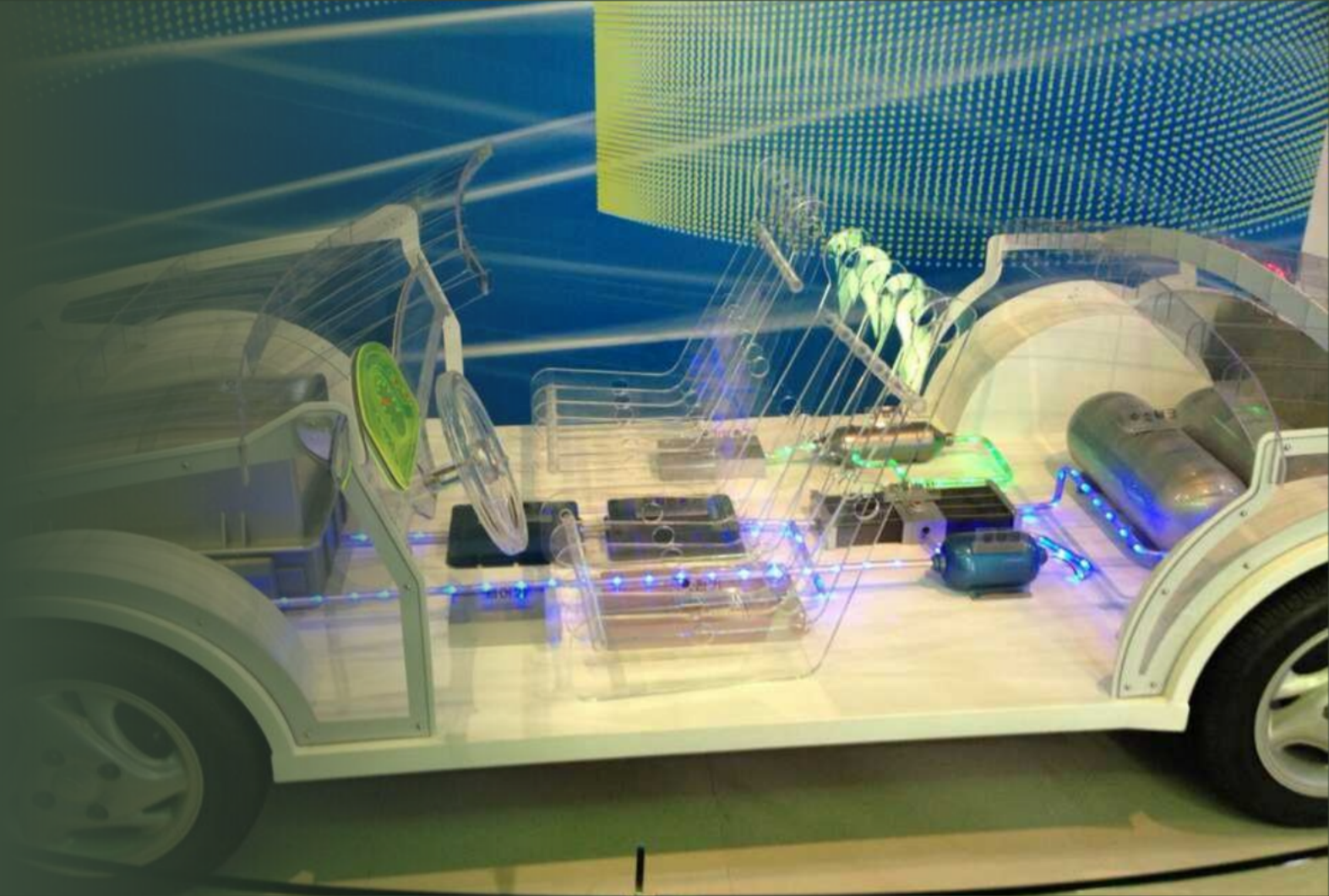
In contrast, traditional feature importance metrics often fail to capture the complex interactions between variables. While they indicate which features are important, they do not explain how each feature contributes to the prediction for individual instances. SHAP analysis addresses this gap by providing a clearer picture of how specific input values affect the output, making the model's predictions more interpretable and accessible to non-technical stakeholders.

Figure 21 — Random Forest: Feature Effects and Direction



Key Insights and Future Directions in Fuel Efficiency Modeling

This project demonstrated that ensemble models, particularly Random Forest and Gradient Boosting, significantly outperform simpler linear regression techniques in predicting vehicle fuel efficiency. However, the study faced limitations due to a small and outdated dataset, which restricts the applicability of the findings. Future research should focus on expanding the dataset to include modern vehicles and exploring additional features such as hybrid technologies and advanced aerodynamics. Enhancements to the modeling framework could also include automated SHAP analysis reporting, allowing for real-time insights into feature importance.



Conclusions & Practical Applications of Fuel Efficiency Modeling



Modeling Fuel Efficiency: A Comparative Analysis of Linear and Ensemble Regression Techniques

An analysis of regression models to evaluate vehicle fuel efficiency, focusing on methodologies and outcomes.

